Chapter 7. Deadlocks. Full solutions

These original derivations correspond to the stable identifiers in the separate exercises PDF.

OS07-E01. Four necessary conditions

1. A and B are exclusive. P1 holds A while requesting B and P2 holds B while requesting A. Neither lock is reclaimable under the original rule. The allocation/request edges form A→P1→B→P2→A.
2. If reclamation safely releases a needed instance and allows a waiter to continue, it breaks this cycle. But reclamation may itself wait for another resource or leave inconsistent data. Its complete protocol must therefore be analyzed rather than treating a preemptible label as a solution.
3. No. In a multiple-instance model another owner can finish, release an instance, and start a sequence that resolves the cycle. A consistent reduction is needed.

Validation criterion. Necessary conditions organize the analysis; proof of deadlock must exclude the relevant progress transitions.

OS07-E02. Safe and unsafe states

1. Total resources are (3,2,1). The original maxima are P1=(2,1,0), P2=(2,2,0), P3=(3,0,1), all within that total. P1 fits Work=(2,1,0); after its completion Work=(3,1,0). P2 fits next, producing (3,2,0). P3 then fits, producing (3,2,1). Thus P1, P2, P3 is safe.
2. Safety simulation gives Work=(2,1,0)+(1,0,0)=(3,1,0). In a real execution, grant Need=(1,1,0) first, reducing Available to (1,0,0). P1 then finishes and returns its original allocation plus the grant, (2,1,0), so Available becomes (3,1,0). The simulation records only this net change; it does not itself change real Available.
3. Changed Max1=(3,1,1) remains legal. P1 cannot start because the third resource is unavailable. P2 can finish and leaves Work=(2,2,0). P1 still lacks the third resource; P3 requires three first-type instances but only two are available. No further reduction is possible, so the changed state is unsafe. This does not prove a current deadlock, since maximum future demands need not already be outstanding.

Validation criterion. Check 0<=Allocation<=Max<=Total, then show every Work and Finish update.

OS07-E03. Victim selection

1. The new lock owner could see an intermediate representation that violates the tree invariant. Exclusion ends before the protected update is complete.
2. Restore a transaction or checkpoint under appropriate isolation: undo must not overwrite another transaction's committed update. Alternatively, construct a complete new copy and atomically publish its root with an appropriate memory-visibility contract. Reclaim the old version only after no reader can still use it. Atomic publication alone does not settle the old version's lifetime.
3. A consecutive-selection cap does not prevent starvation: selecting A, B, A, B indefinitely never selects either victim twice in a row, yet either can be rolled back infinitely often. Counts or aging need an additional fairness and progress argument.

   A conservative provable rule fixes an order for the processes. In P's turn, protect P from victimization; safely roll every other unfinished process back to a consistent checkpoint, reclaim its resources, and pause its further requests until P finishes. Rollback and reclamation take finite time by assumption. No other process then holds resources, and P's maximum demand is within total capacity, so its remaining requests can be met. Fair grants, fair CPU scheduling, and finite remaining work let P finish. Resume and protect the next process in the fixed order, repeating until all complete. With n processes, each suffers at most n−1 rollbacks before its own turn, so no process is victimized forever. This guarantee is costly; it does not follow if rollback is unsafe, outside holders retain resources indefinitely, or new interfering work keeps arriving.

Validation criterion. Recovery integrity and victim fairness require separate arguments.

OS07-E04 Resource-graph cycle

1. Requests P1→R2, P2→R1. Assignments R1→P1, R1→P3, R2→P2. No instance is initially free.
2. Cycle P1→R2→P2→R1→P1 exists. Resources are exclusive; P1/P2 hold and wait; there is no forced preemption; circular wait appears. Circular wait already implies hold-and-wait.
3. P3 can finish and release R1. P2 then takes it, finishes and returns R2. P1 then finishes. Thus this multiple-instance cycle does not establish deadlock. The completion argument assumes P3 is scheduled and actually returns its resource.

OS07-E05 Banker request

1. (1,0,2)≤Need1(1,2,2) and ≤Available(3,3,2). This only authorizes trial.
2. Trial Available=(2,3,0), Allocation1=(3,0,2), Need1=(0,2,0); all other rows unchanged.
3. Work(2,3,0). P1 Need(0,2,0) fits→(5,3,2). P3 Need(0,1,1) fits→(7,4,3). P4 Need(4,3,1) fits→(7,4,5). P0 Need(7,4,3) fits→(7,5,5). P2 Need(6,0,0) fits→(10,5,7). Sequence ⟨P1, P3, P4, P0, P2⟩ certifies safety and the request can commit.
4. Restore Available, Allocation_i, and Need_i to the pre-request state before waiting. No partial debit/allocation may remain. The simulated completion adds existing Allocation, not Need.

OS07-E06 Multi-instance detection and recovery

1. All Allocation rows are nonzero, so initial Finish all false. Work=(0,0,0). P0's zero request fits, giving Work=(0,1,0), Finish=(true, false, false, false, false). No other request fits.
2. Under the fixed-snapshot and waiting assumptions in the problem, P1/P2/P3/P4 remain detected deadlocked holders; P0 can complete in the model. Simulation alone does not show that P0 has already completed. If requests can cancel, resources can be released early, or further requests can arise, this snapshot conclusion does not directly guarantee the actual subsequent execution.
3. If P0 has not actually finished, safely aborting P1 returns only (2,0,0), so actual Available=(2,0,0). Restart Work at that value: simulate P0→(2,1,0), P3→(4,2,1), P2→(7,2,4), P4→(7,2,6), checking each current Request≤Work before returning its Allocation. This certifies a survivor completion sequence; simulated Finish does not release real resources. Available=(2,1,0) immediately after abort is valid only if P0 has already completed in the real execution.
4. Reclaiming exclusion does not undo partial updates. Roll back the victim to a consistent transaction/checkpoint before survivors observe its data; preserve resource conservation and prevent unlimited repeated victimization separately.

OS07-E07

Graph and criterion. Assign R1a→P1, R1b→P3, R2a→P2. Requests are P1→R2 and P2→R1. The cycle is P1→R2→P2→R1→P1. R1 has two instances, so the cycle alone does not establish deadlock; P3 can finish first.

Simulated reduction. P3: (0,0)≤(0,0), return (1,0), Work=(1,0). P2: (1,0)≤(1,0), return (0,1), Work=(1,1). P1: (0,1)≤(1,1), return (1,0), Work=(2,1). No process remains. If only this calculation has run, live Available is still (0,0).

Actual execution. P3 finishes→(1,0); grant P2 one R1→(0,0); P2 finishes and returns both types→(1,1); grant P1 one R2→(1,0); P1 finishes→(2,1). Each actual return includes the newly granted request as well as previous holdings.

Modified case. Start Work=(0,0). P1's (0,1) fails in component 2; P2's (1,0) fails in component 1; P3's (0,1) also fails in component 2. All hold resources and none can finish first, so the deadlocked set is {P1,P2,P3}. Adding P3→R2 blocks the owner that could previously release R1b. The new cycle P3→R2→P2→R1→P3 shares R2 with the original cycle.

OS07-E08

Initial Need: (2,2), (1,1), (2,1). P2's (0,1) passes both checks. Trial: Available=(1,0), Allocation2=(1,2), Need2=(2,0). P0 fails A/B, P1 B, P2 A. No candidate: Work=(1,0), Finish=F/F/F; reject.

Restore Available=(1,1), Allocation2=(1,1), Need2=(2,1). Restoring only Available makes the total (4,4), adding a nonexistent B. Need2 must also restore Max2=Allocation2+Need2.

Restart from the initial state. P1's (0,1) trial gives Available=(1,0), Allocation1=(1,2), Need1=(1,0).

Simulated finish	Old Work	Need	Return Allocation	New Work	Finish
P1	(1,0)	(1,0)	(1,2)	(2,2)	F/T/F
P0	(2,2)	(2,2)	(1,0)	(3,2)	T/T/F
P2	(3,2)	(2,1)	(1,1)	(4,3)	T/T/T

Commit the trial. Actual Available remains (1,0), not the final Work.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

At U, Available=(0,1), Need=(1,2),(1,1),(2,1), and Allocation=(2,0),(1,1),(1,1). One extra A gives Work=(1,1), allowing P1→P0→P2 with Work=(2,2)→(4,2)→(5,3), the new Total. One extra B gives (0,2); every process still lacks A. Any number of additional B instances leaves this first-step obstruction intact.

With no extra resources, the specified actual execution is:

Action	Available after grant	Return on completion	Available after return
P0 takes (0,1)	(0,0)	(2,1)	(2,1)
P1 takes (1,0)	(1,1)	(2,1)	(3,2)
P2 takes (1,0)	(2,2)	(2,1)	(4,3)

Each request fits Need; each return includes the new grant. At U, P0's maximum Need=(1,2) does not fit Work=(0,1). Thus P0→P1→P2 is an actual completion order, not a safe sequence. It works because P0 needs less: (0,1).

References and attribution

Lin Cing, handwritten operating systems notes. Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapter 7.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.
https://github.com/xinhi6716/computer-science-self-study